Checkpoint 3 — Performance Audit & Concurrency Report

Team Repository Submission: `https://github.com/pattanankm/2Handy.git`

THROUGHPUT (RPS)	P95 LATENCY	P99 LATENCY	ERROR RATE
186.8	**37 ms**	**100 ms**	**0.00%**
Stable Continuous RPS	Goal: < 200 ms (PASS)	Goal: < 500 ms (PASS)	Goal: < 1% (PASS)

1. Executive Summary & Benchmark Compliance

This audit evaluates the production readiness of the 2Handy Web API under high concurrent write loads and heavy database query stress, adhering strictly to the **Checkpoint 3 Specification**. The system demonstrated flawless concurrency control without race conditions or lost updates, eliminated all N+1 query bottlenecks via SQL/projection optimizations, and exceeded all performance thresholds under a sustained 3-minute benchmark.

Metric	Specification Target	Actual Result	Status	Margin of Safety
Virtual Users (VUs)	Minimum 50 Concurrent VUs	**50 VUs**	PASS	100% Target Met
Test Duration	3-Minute Continuous Load	**180 Seconds (3m 00s)**	PASS	Sustained Load Verified
p95 Response Latency	< 200 ms	**37 ms**	PASS	81.5% Faster than Limit
p99 Response Latency	< 500 ms	**100 ms**	PASS	80.0% Faster than Limit
HTTP Error Rate	< 1.00%	**0.00% (0 / 33551 requests)**	PASS	Zero Server Exceptions
Throughput Stability	Stable RPS across run	**186.85 req/s**	PASS	Zero Thrashing / Consistent

2. Automated Load Testing Breakdown (50 VUs Sustained)

The benchmark simulated a realistic e-commerce traffic pattern mixing read-heavy browsing tasks with write-intensive transactional checkouts.

Throughput Timeline (Requests Per Second over 180s)



Endpoint / Transaction	Method	Requests	RPS	p50	p95	p99	Failures
Read: Get Product Detail	`GET`	5135	28.60	5 ms	**33 ms**	98 ms	0 (0.00%)
Read: List Orders (1 SQL)	`GET`	6362	35.43	8 ms	**35 ms**	99 ms	0 (0.00%)
Read: List Products (Projected)	`GET`	7727	43.03	6 ms	**34 ms**	95 ms	0 (0.00%)
Read: Product Reviews	`GET`	3894	21.69	5 ms	**36 ms**	100 ms	0 (0.00%)
Read: Product Summaries ($lookup)	`GET`	5126	28.55	8 ms	**36 ms**	100 ms	0 (0.00%)
Setup: Fetch Initial Products	`GET`	50	0.28	26 ms	**50 ms**	51 ms	0 (0.00%)
Write: Create Order (Dual-DB)	`POST`	2636	14.68	12 ms	**62 ms**	150 ms	0 (0.00%)
Write: Create Product (OCC v1)	`POST`	2621	14.60	5 ms	**34 ms**	110 ms	0 (0.00%)
Aggregated (Total)	**ALL**	**33551**	**186.85**	**7 ms**	**37 ms**	**100 ms**	**0 (0.00%)**

3. Concurrency Control & Race Condition Prevention Audit

To satisfy Section 2.1 of the CP3 Specification, the 2Handy architecture implements and proves both concurrency control paradigms:

A. Optimistic Concurrency Control (OCC) in MongoDB

- **Mechanism:** Every product document in MongoDB maintains an atomic integer `version` field. Product updates and purchases execute via atomic `find_one_and_update` requiring `version == expected_version` and incrementing `version` via `$inc: {"version": 1}`.
- **HTTP Status Mapping:** When a concurrent write collides, the server returns **HTTP 409 Conflict** (or **HTTP 412 Precondition Failed** when using stale `If-Match` headers).
- **Dual-Database Integrity:** When buying an item in `POST /api/v1/orders`, if MongoDB OCC rejects the sale because another user purchased the last item, PostgreSQL automatically rolls back the order transaction. Zero orphan rows remain.

B. Pessimistic Concurrency Control (PCC) in PostgreSQL

- **Mechanism:** Order state transitions (e.g. `POST /api/v1/orders/{id}/complete`) and critical user updates acquire an explicit row lock inside a database transaction boundary using `SELECT ... FOR UPDATE`.
- **Blocking & Serialization:** Concurrent requests are blocked until the lock-holding transaction commits. Subsequent requests inspect the updated status, detect the order is no longer `pending`, and safely abort with **HTTP 409 Conflict**.

C. Automated Concurrency Verification Script (`test_concurrency.py`)

The dedicated test script simulates 20 threads released concurrently at the exact same millisecond using `threading.Barrier(20)`:

```
$ python test_concurrency.py

📡 Connected to active API server at http://127.0.0.1:8000
======================================================================
🚀 [TEST 1] Concurrency Test: 20 concurrent users buying LAST item
======================================================================
📦 Created single-stock product: ID=6adba16ef860c6a25fd160e2
👤 Using test user: ID=5777
⚡ Firing 20 simultaneous requests to POST /api/v1/orders...
📊 Results -> Success (201): 1 | Conflicts (409): 19 | Others: 0
✅ [TEST 1 PASSED] Flawless OCC Race Condition Prevention verified!
   • Exactly 1 user bought the product (HTTP 201)
   • Exactly 19 users received HTTP 409 Conflict
   • PostgreSQL has exactly 1 order_item record (Zero ghost orders)
   • MongoDB product marked 'sold' with version bumped to 2

======================================================================
🚀 [TEST 2] Concurrency Test: 20 concurrent users completing SAME order
======================================================================
📋 Created pending order: ID=1016
⚡ Firing 20 simultaneous requests to POST /api/v1/orders/1016/complete...
📊 Results -> Success (200): 1 | Conflicts (409): 19 | Others: 0
✅ [TEST 2 PASSED] Flawless Pessimistic Row Lock (SELECT FOR UPDATE) verified!
   • Exactly 1 request completed the order (HTTP 200)
   • Exactly 19 requests received HTTP 409 Conflict
   • Order status in PostgreSQL is 'completed'
```

```
======================================================================
🎉 ALL CONCURRENCY VERIFICATION TESTS PASSED SUCCESSFULLY! (100% PASS)
======================================================================
```

4. Query Projection & N+1 Bottleneck Elimination Audit

In accordance with Section 2.2 of the CP3 Specification, all read-only endpoints were audited and refactored to guarantee **O(1) database queries** with explicit projection:

Endpoint	Query Strategy	Round Trips	Projection Optimization
`GET /api/v1/orders`	CTE + `LEFT JOIN` + `jsonb_agg`	**1 SQL Query**	Aggregates order items as JSON directly in PostgreSQL; avoids ORM lazy loading.
`GET /api/v1/orders/{id}`	Parametrized CTE + `jsonb_agg`	**1 SQL Query**	Extracts only order header and child item tuples in one disk fetch.
`GET /api/v1/products`	Indexed Mongo `find()`	**1 Mongo Find**	Omits heavy internal metadata; projects only `name, category, status, attributes, version`.
`GET /api/v1/products/summary`	Pipeline with `$lookup`	**1 Aggregation**	Calculates `review_count` and `avg_rating` directly in MongoDB engine with zero application loops.
`POST /api/v1/orders`	Batch read with `$in`	**1 Mongo Find**	Fetches all item availability states in a single query before OCC write operations.